062102 - Applied Statistics

Polytechnic University of Milan

A.Y. 2025/26 *class notes*

Authors: Guflie

Professor: Massi Michela Carlotta

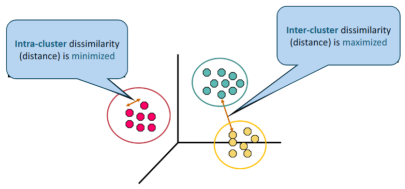
Contents

1 Clustering **3**
1.1 K-means algorithm 4
1.2 Hierarchical clustering 5
1.3 Evaluating cluster quality 6
1.3.1 Within Cluster Sum of Square (WCSS) 6

1 Clustering

This class is a type fo *unsupervised learning* in which we assume that the data has some kind of unkwon structure, and we wan to figure it out. To do that we try to accomplish two things:

- minimize distance between datapoints of the same cluster
- maximize distance between $\neq$ clusters



The clustering output is really subjective and depend what we're trying to achieve and our conception of **similarity**. We have to define two elements:

Distance metric : tells what makes two object similar/dissimilar

Clustering algorithm : procedure to compute groups/clusters

Distance metrics

The definition of Distance Metrics depends mostly on the type of the data features. Some common ones are:

- **Euclidean distance**: $d_E(x,q) = \sqrt{\sum_{i=1}^{n}(x_1 - q_1)^2}$
- **Squared Euclidean Distance**: $d_{E^2}(x,q) = (d_E(x,q))^2 \rightarrow$ with respect to d_E it gives more emphasys to outliners (distance is squared)
- **Standardize Euclidean Distance**: $d_{SE}(x,q) = \sqrt{\frac{1}{s_i^2}(x_i - q_i)} \rightarrow$ weights each dimension by something inverse to its variability, favouring those dimensions that doens't change too much
- **Chebychev Distance**: $d_{\max}(x,q) = \max_i |x_i - q_i|$ (just the max, not really useful)
- **Manhattan Distance**: $d_M(x,q) = \sum_{i=1}^{n} |x_i - q_i| \rightarrow$ gives the # step to reach a destination in a grid by moving up and down, lot of usage in many fields
- **Correlation-based Distance**: $d_R(x,q) = 1 - \frac{s_{x,q}}{\sqrt{s_x}\sqrt{s_q}} = 1 - \frac{\sum_i (x_i - \bar{x})(q_i - \bar{q})}{\sqrt{\sum_i (x_i - \bar{x})^2 \sum_i (q_i - \bar{q})^2}} \rightarrow$ this measures allow to get an idea about the distance between feature trends distance

Highly suggested to use d_R in the project

To compare the datapoint features, we must be sure that everyone is using the same measurement unit.

Clustering algorithms

There are several way to categorize these methods:

Hard clustering : each datapoint can be only in one cluster

Soft clustering : each datapont might be in more clusters at the same time

Flat clustering : all clusters are at the same level

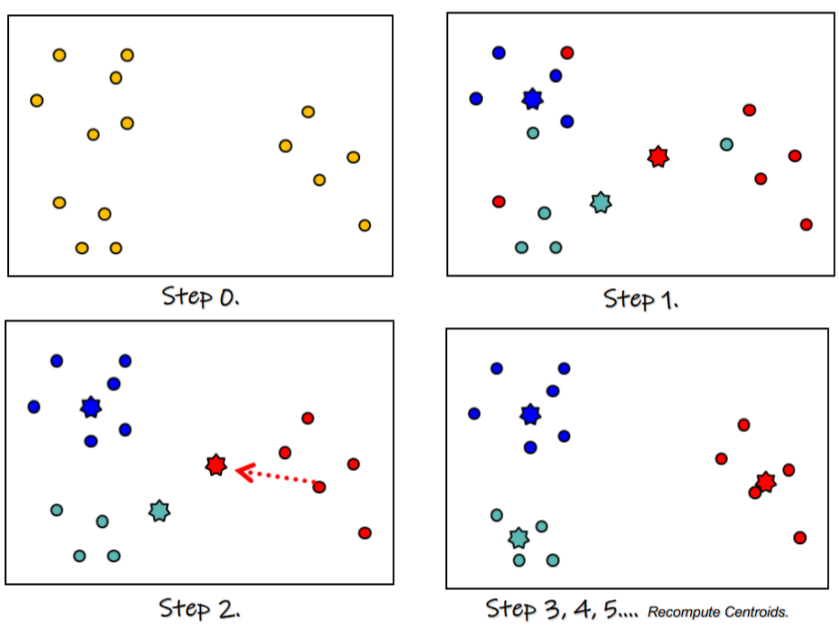
Hierachical clustering : the cluster are under some hierarchy (dendogram)

1.1 K-means algorithm

In this algorithm we priorly fix the number of clusters k, starting from an initial assignment and itereatively moving datapoints between the clusters.

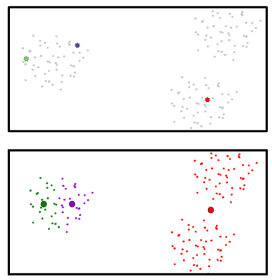
1. define $k \to C_1 \dots C_k$ clusters
2. randomly assign each point to some cluster C_i and compute the centroid for that set of points
3. re-assign each point to the closest centroid cluster, ending up possibly with new point in each cluster
4. recompute the centroids, and iterate from *step 2*

We iterate unitl the algorithm converges, e.g. the points won't change cluster.

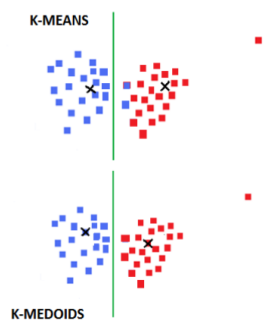


Limitations of this algorithm:

- the algorithm output strongly depends on the initial random assignment; if there is some structure probably the algorithm will converge at the same result, but if this structure is not so evident the outcome might be very sensitive.
  To reduce this influence we could try out multiple seeds in parallel and choose at the end the best result.



- we have to choose k in advance, maybe without clear idea of we're doing
- the distance used affects a lot the centroid: if we use *Euclidean Distance* our computation is going to be very sensitive to outliers We can resolve this by choosing another method to compute distances: instead of using



  d_E we use the **mode**, which are more robuts to utliers $\rightarrow$ the *medoid* is also more reasonable to use since we'll end up with something that is a real point of the cluster (like the most representative point).

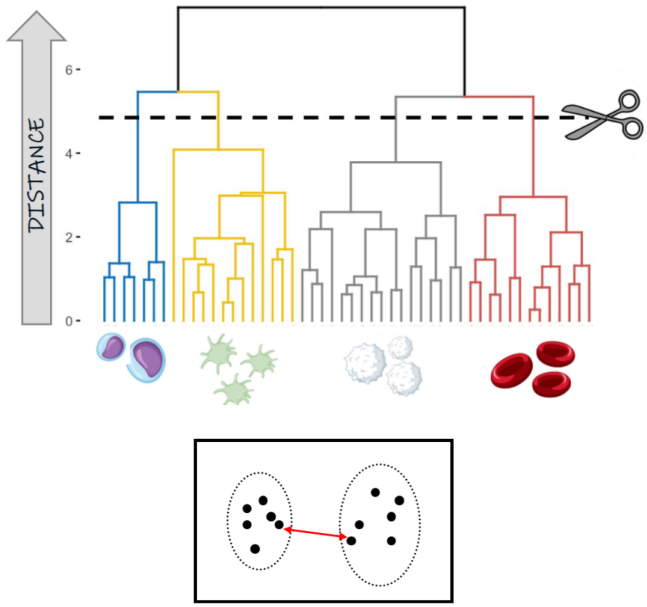
- with strange-shape clusters, K-means makes a very bad work (how can we know that in advance? You can't) $\rightarrow$ k-means fits well spherical/hellipsoidal datashapes

1.2 Hierarchical clustering

This method builds a hierarchy among the clusters using a **Distance Matrix D**, which contains the distance between each pair of samples. To stop the procedure we have to define a **cutting criterion** to stop the construction of the **dendogram** := representation of your cluster. The algorith can both start by considering each datapoint in a $\neq$ cluster and then merging them, or start from a one big cluster and progressively split it.

1. start from considering each datapoints as a cluster per se
2. merge the two closer clusters and take a "center" (linkage methods)
3. recompute the set of points and iterate starting from 1

By merging the cluster D become smaller at each iteration.
By using this algorithm we can also highligh noisy datapoints and anomalies: if a point is added very late in the cluster, maybe it's just an anomaly $\rightarrow$ this method can also be used for anomaly detection.



Linkage methods
How do we computer the "center" between build-up clusters?

Single link : cluster-distance is based on the two closest points from each cluster

Average link : cluster-distance is based on the average position of the cluster datapoints

Complete link : cluster-distance is based on the most distant points from each cluster

There is no rule of thumb, start with the average and adjust things.

1.3 Evaluating cluster quality

How do we eavulate if our clusters make sense if we don't have an output reference?

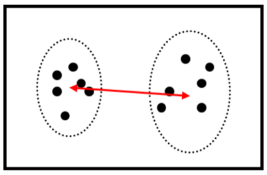
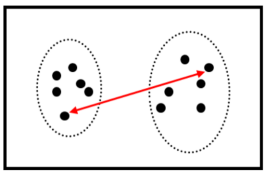
1.3.1 Within Cluster Sum of Square (WCSS)

WCSS measures the variability of the observations within each cluster.

$$WCSS = \sum_{i} \sum_{p \in C_i} dist(p, C_i)^2 \tag{1}$$

The smaller our clusters are, the lower(better) the value of WSS. However by following this metric, we get that the best result is we each datapoint form a cluster per-se. So we don't want to minimize this, otherwise is useless; we need just to find a point in which it doesn't decreases a too much.

With this method we're also looking at how similar points within the same cluster are, without considering the other goal of clustering: distance between $\neq$ clusters.

Solhouette Score This metrics takes into account both the distance between points in the same cluster and the distance from other cluster's points.

1. $\forall$